PYTHON FOR DATA SCIENCE · LESSON 30 OF 31

Automation: Scheduled Scripts, Reports and Email

Turn a manual report into a script that runs on a schedule, reads the data, builds a summary, logs every run and sends the result by email without storing a password in the code.

Gap-fill lesson | 2026 | Intermediate | 120 min

By the end of this lesson you can

- Write a script with a single entry point that builds a report from a CSV file
- Schedule a script with cron (Linux and macOS) or Task Scheduler (Windows)
- Send a report by email using smtplib, reading credentials from the environment
- Log each run and report failures clearly

Datasets used

`daily_sales.csv`

Why automate a report

Most analytics teams have a report that someone builds by hand every Monday: export, filter, pivot, paste into an email. It takes an hour, it is easy to get wrong, and it stops when that person is on leave. A script does the same steps the same way every time, and a scheduler runs it without anyone remembering.

CASE STUDY

The Monday sales email

Every Monday at 07:00 the regional manager needs last week's units sold, the best and worst days, and the total, in an email. The analyst builds this by hand from a CSV export. The task is to write one script that does it, runs unattended, logs its work and tells someone when it fails.

Step 1: a script with one entry point

Put the logic in functions and call them from a `main()` function. This makes the code testable and lets a scheduler run it with one command.

Python

```
import pandas as pd

def load_sales(path):
    """Read the daily sales file and check the columns we need."""
    df = pd.read_csv(path, parse_dates=["date"])
    missing = {"date", "units_sold"} - set(df.columns)
    if missing:
        raise ValueError(f"Missing columns: {sorted(missing)}")
    return df

def weekly_summary(df, week_start, week_end):
    """Summarise one week: total, best day and worst day."""
    week = df[(df["date"] ⩾ week_start) & (df["date"] ⩽ week_end)]
    if week.empty:
        raise ValueError(f"No rows between {week_start} and {week_end}")
    best = week.loc[week["units_sold"].idxmax()]
    worst = week.loc[week["units_sold"].idxmin()]
    return {
        "total_units": int(week["units_sold"].sum()),
        "days": len(week),
        "best_day": best["date"].date().isoformat(), "best_units": int(best["units_sold"]),
        "worst_day": worst["date"].date().isoformat(), "worst_units":
int(worst["units_sold"]),
    }

summary = weekly_summary(load_sales("daily_sales.csv"), "2025-01-06", "2025-01-12")
print(summary)
```

OUTPUT

```
{'total_units': 2196, 'days': 7, 'best_day': '2025-01-11', 'best_units': 338,
'worst_day': '2025-01-06', 'worst_units': 297}
```

Each function does one job, so you can test `weekly_summary` on a small table without a file. The script raises a clear error when a column is missing or a week has no data, rather than producing a silent wrong report.

Step 2: log every run

Print statements disappear when a scheduled job runs in the background. The `logging` module writes timestamped messages to a file, so you can check what happened last Monday.

Python

```python
import logging
import io

stream = io.StringIO()                       # in a real script: filename="report.log"
logging.basicConfig(stream=stream, level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s", force=True)
log = logging.getLogger("weekly_report")

log.info("starting run")
log.info("loaded %d rows", 365)
log.warning("two days had zero sales")
log.error("email step failed: %s", "connection refused")
print(stream.getvalue())
```

OUTPUT

```
2026-11-11 19:10:15,148 INFO starting run
2026-11-11 19:10:15,148 INFO loaded 365 rows
2026-11-11 19:10:15,148 WARNING two days had zero sales
2026-11-11 19:10:15,148 ERROR email step failed: connection refused
```

Use levels deliberately: `INFO` for normal progress, `WARNING` for something unexpected that did not stop the run, `ERROR` for a failure that did.

Step 3: send the email without storing a password

Never write a password into the script. Read it from an environment variable, which you set in the scheduler or the server's secret store (Module 6 of the course explains environment variables). This block is shown as code and is **not run**, because it needs a mail server.

Python

```python
import os
import smtplib
from email.message import EmailMessage

def send_report(body, to_addr):
    msg = EmailMessage()
    msg["Subject"] = "Weekly sales summary"
    msg["From"] = os.environ["REPORT_FROM"]
    msg["To"] = to_addr
    msg.set_content(body)
    with smtplib.SMTP_SSL(os.environ["SMTP_HOST"], 465) as server:
        server.login(os.environ["SMTP_USER"], os.environ["SMTP_PASSWORD"])
        server.send_message(msg)
```

::::warn Passwords and personal email Do not put the password in the script, the repository or a log message. Many mail providers now require an app password or OAuth rather than your normal login. Check your provider's current rules before you automate. ::::

Step 4: schedule it

Scheduling is the operating system's job, not Python's.

System	Tool	Example entry
Linux, macOS	`cron` (edit with `crontab -e`)	`0 7 * * 1 /usr/bin/python3 /home/analyst/weekly_report.py >> /home/analyst/report.log 2>&1`
Windows	Task Scheduler	Trigger: weekly, Monday 07:00. Action: `python.exe` with the script path.

The cron line runs at 07:00 every Monday (`0 7 * * 1`) and sends both output and errors to the log file (`>> ... 2>&1`). Use full paths to Python and the script, because a scheduler does not load your shell's settings.

Step 5: handle failures

A scheduled job that fails silently is worse than no job. Wrap the main steps so that any failure is logged and reported.

Python

```python
import logging, io
stream = io.StringIO()
logging.basicConfig(stream=stream, level=logging.INFO, format="%(levelname)s %(message)s",
force=True)
log = logging.getLogger("weekly_report")

def run_report(path, start, end):
    try:
        df = pd.read_csv(path, parse_dates=["date"])
        log.info("loaded %d rows", len(df))
        return weekly_summary(df, start, end)
    except FileNotFoundError:
        log.error("input file not found: %s", path)
    except ValueError as e:
        log.error("report failed: %s", e)
    return None

result = run_report("daily_sales.csv", "2030-03-01", "2030-03-07")   # outside the data:
fails, logged
print("result:", result)
print(stream.getvalue())
```

OUTPUT

```
result: None
INFO loaded 1461 rows
ERROR report failed: No rows between 2030-03-01 and 2030-03-07
```

The function returns `None` on failure, and the main program turns that into a non-zero exit code. A scheduler can then notify you, because a job that exits with an error is a failed job.

::::tip Test the script before scheduling it Run it by hand with last week's file, check the output against a manual calculation, then run it once through the scheduler at a time when you can watch the log. ::::

Step 6: a checklist for every automated report

1. The input file's columns and date range are checked before use.
2. Every run writes a log line with its start, row count and result.
3. Failures produce an error message and a non-zero exit code.
4. Credentials come from the environment, not the code.
5. The output is saved (for example to a file or a shared folder) as well as emailed, so it can be checked later.
6. A person owns the schedule and reads the log on a regular basis.

::::key What to remember - Put the logic in functions with one `main()` entry point, so scripts can be tested and scheduled. - Log to a file with levels; scheduled jobs run where you cannot see the screen. - Keep passwords out of code; read them from environment variables or a secret store. - Schedule with cron or Task Scheduler, using full paths. - A failed job must produce an error and a non-zero exit code. ::::

Quick quiz

5 questions · tap an answer to check it

1. Why use `logging` rather than `print` in a scheduled script?

- **A** print is slower
- **B** Scheduled jobs run without a screen, and logging writes timestamped messages to a file
- **C** logging is required by cron
- **D** print cannot show numbers

Answer: B. Output from background jobs has to be recorded somewhere you can read later.

2. Where should an SMTP password be stored?

- **A** In the script, in a variable called PASSWORD
- **B** In the repository, in a config file
- **C** In an environment variable or secret store, read at run time
- **D** In the email body

Answer: C. Keeping secrets out of code stops them leaking through version control and logs.

3. What does the cron expression `0 7 * * 1` mean?

- **A** Every hour at 1 minute past 7
- **B** At 07:00 every Monday
- **C** Seven times a day
- **D** On the first day of each month

Answer: B. Fields are minute, hour, day of month, month and day of week (1 = Monday).

4. `weekly_summary` raises an error when the week is empty. Why is that better than returning zeros?

- **A** Zeros are not numbers
- **B** A silent zero can look like a real result; an error makes the failure visible
- **C** Errors are faster
- **D** It avoids logging

Answer: B. A report that is wrong without warning is more harmful than one that fails loudly.

5. A scheduled job fails at night. What should it do so that someone notices?

- **A** Retry forever
- **B** Print the error to the console
- **C** Log the error and exit with a non-zero code that the scheduler can report
- **D** Send the email with an empty body

Answer: C. A failing job should be visible to whoever owns it.

EXERCISE **A daily summary job**

Write a script `daily_summary.py` that reads `daily_sales.csv`, computes the total units and the average for the last 7 days of the file, writes one line per run to `summary.log` (timestamp, rows, total, average), and exits with code 1 if the file is missing. Schedule it on your machine to run once a day and check the log the next morning.

Show solution (worked answer)

Hints: find the last date with `df["date"].max()`, then filter `date > max - 7 days`. Use `logging.basicConfig(filename="summary.log", ...)`. For the exit code, `import sys` and call `sys.exit(1)` in an `except FileNotFoundError` block.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.